



Frontend ↔ Backend Wiring Prompt

A complete implementation specification for connecting all disconnected and partially-connected frontend components to the FastAPI backend.

Files: Editor.tsx · floating-dock.tsx · page.tsx (home)

New components: Chat.tsx · ProblemList.tsx · WeeklyInsight.tsx

3	5	3
files to fix	endpoints to wire	new components

How to use this document

This document contains a single structured prompt you can paste directly into any AI coding assistant (Claude, Cursor, GPT-4, etc.) to wire the entire Kdo frontend to its FastAPI backend in one pass. The prompt is split into five numbered tasks. Each task targets a specific file, specifies the exact endpoint to call, defines the expected request/response shape, and describes the UI behavior required. You may run all five tasks together or paste them one at a time.

Important: The backend base URL is assumed to be `http://localhost:8000`. Replace with your deployed URL before going to production. All streaming responses from `POST /chat` use plain `text/plain` chunked transfer — not SSE.

The complete wiring prompt

Copy everything inside the box below and paste it as a single message to your AI assistant. The prompt includes all context the model needs — file names, endpoint contracts, streaming patterns, state management, and exact component behavior.

CONTEXT

You are working on Kodo — an AI-powered coding mentor built with Next.js 16 (App Router) on the frontend and FastAPI on the backend. The backend is running at `http://localhost:8000`. The frontend currently has no API integration whatsoever. Your job is to wire the frontend to the backend completely. Do not change any existing UI styling, layout, or animation. Only add data-fetching logic, state, and API calls.

BACKEND API REFERENCE

Method	Path	Purpose
POST	<code>/session/start</code>	Returns a personalized session opener based on user memory.
POST	<code>/chat</code>	Streams Kodo's reply. May return an action to open a problem.
GET	<code>/problem/list</code>	Returns array of all problems with id, title, difficulty, tags.
GET	<code>/problem/{id}</code>	Returns full problem: description, examples, test_cases, starter_code.
POST	<code>/problem/submit</code>	Submits code attempt with pass/fail. Stores behavioral memory.
GET	<code>/insight/weekly</code>	Returns weekly learning summary string for the user.

TASK 1 — Create `src/lib/api.ts` (API client)

Create a new file `src/lib/api.ts`. This is the single source of truth for all backend calls. It must export the following functions. Use the user_id `"user_001"` as a hardcoded constant for now (auth will come later).

```

// api.ts — full function signatures to implement

// src/lib/api.ts

const BASE = "http://localhost:8000";

const USER_ID = "user_001";

// 1. Session start — returns plain string greeting
export async function startSession(): Promise<string>

// 2. Chat — streams response, calls onChunk per token
// If response JSON contains action.type === "open_problem",
// call onAction({ type: "open_problem", problem_id: string })
export async function chat(
  query: string,
  problemId?: string,
  currentCode?: string,
  onChunk: (t: string) => void,
  onAction?: (a: ChatAction) => void
): Promise<void>

// 3. Problem list
export async function getProblemList(): Promise<ProblemSummary[]>

// 4. Single problem
export async function getProblem(id: string): Promise<Problem>

// 5. Submit solution
export async function submitProblem(
  problemId: string, code: string,
  passed: boolean, attempts: number
): Promise<SubmitResult>

// 6. Weekly insight
export async function getWeeklyInsight(): Promise<string>

```

Streaming implementation note: `POST /chat` returns `text/plain` chunked transfer, not SSE. Read it with `response.body.getReader()` and decode each chunk with `TextDecoder`. Before streaming, attempt to parse the first chunk as JSON — if it has `action.type === "open_problem"` call `onAction` and stop. Otherwise stream all chunks to `onChunk`.

TASK 2 — Create `src/components/ui/Chat.tsx`

Create a new client component `Chat.tsx`. This is the core Kodo interface. It must:

- On mount, call `startSession()` and display the returned greeting as Kodo's first message.
- Render a scrollable message list. Each message has a `role` of `"kodo"` or `"user"` and a `content` string.
- Render a text input and submit button at the bottom. On submit, append the user message, then call `chat(query, problemId, currentCode, onChunk, onAction)`. Stream Kodo's reply token-by-token into a new message using `onChunk` by updating the last message's content in state.
- Accept props: `problemId?: string` and `currentCode?: string` — these are passed down from Editor when a problem is active.
- If `onAction` fires with `type: "open_problem"`, call the prop `onOpenProblem(problem_id: string)` so the parent can navigate to that problem.
- Show a typing indicator (animated dots) while streaming.
- Do not change any existing color variables or fonts — use the project's existing Tailwind classes `bg-[#3D1515]`, `text-[#EFEDE3]` etc.

TASK 3 — Fix `src/components/ui/Editor.tsx`

Modify the existing `Editor.tsx` to become a fully connected problem-solving environment. The component currently runs code in Pyodide but has no awareness of problems or the backend.

- Add prop `problemId?: string`. When provided, fetch the full problem on mount with `getProblem(problemId)` and store it in state.
- Add a left panel (resizable or fixed split) that shows the problem `title`, `description`, `examples`. Initialize the Monaco editor with `problem.starter_code` when a problem is loaded.
- Track `attemptCount` in state. Increment on every RUN click.
- After Pyodide execution completes, compare stdout against each entry in `problem.test_cases`. Set `passed = true` if all test case outputs match.
- Add a SUBMIT button (only visible when `passed === true`). On click, call `submitProblem(problemId, code, passed, attemptCount)` and show the returned `feedback` string in the console output.
- Pass `currentCode` and `problemId` as props to the component so Kodo can see what the user is working on.

```
// Editor.tsx — target layout (3-column)

// Editor.tsx — new layout structure
return (
  <div className="flex h-full w-full">
    </* LEFT: Problem statement (1/3 width) */>
    <ProblemPanel problem={problem} />
    </* CENTER: Monaco + terminal (1/3 width) */>
    <CodePanel code={code} output={output} ... />
    </* RIGHT: Kodo chat panel (1/3 width) */>
    <Chat
      problemId={problemId}
      currentCode={code}
      onOpenProblem={({id} => setProblemId(id)}
    />
  </div>
)
```

TASK 4 — Create `src/components/ui/ProblemList.tsx` + wire `workspace/page.tsx`

Create a new component `ProblemList.tsx` and update `workspace/page.tsx` to manage routing between the problem list and the editor.

- `ProblemList.tsx` fetches `GET /problem/list` on mount and renders each problem as a card showing title, difficulty badge, and tags.
- On card click, call the prop `onSelect(problem_id: string)` to navigate into the editor for that problem.
- Difficulty badges: `easy` = green, `medium` = amber, `hard` = red. Match the project's color palette.
- `workspace/page.tsx` should manage a `selectedProblemId` state. When null, show `ProblemList`. When set, show `Editor` with that problem ID as a prop.
- Add a back button inside the editor header that resets `selectedProblemId` to null, returning to the problem list.

TASK 5 — Wire session start + create `src/components/ui/WeeklyInsight.tsx`

Two remaining gaps: the session-start greeting must fire on app load, and the weekly insight needs a display component.

- In `src/app/layout.tsx` (or a new client wrapper), call `startSession()` once on mount. Store the result in a Zustand store `useKodoStore` under the key `sessionGreeting`. The `Chat` component should read this as its first message instead of calling `startSession` itself.

- Create `src/components/ui/WeeklyInsight.tsx`. It fetches `GET /insight/weekly?user_id=user_001` on mount and renders the returned string as a styled card. Show a skeleton loader while fetching.
- Add `WeeklyInsight` to the workspace page, below the problem list — visible only when no problem is selected.
- In `floating-dock.tsx`, import `usePathname` from `next/navigation` and pass `highlight={pathname === item.href}` to each `IconContainer` to show the active route.

FINAL INSTRUCTION TO AI ASSISTANT

Complete all five tasks above. For each file you modify or create, output the complete file — not a diff, not a snippet. Preserve all existing Tailwind classes, animation logic (GSAP, Framer Motion), and Three.js components exactly as they are. Only add the wiring described. Use TypeScript strict mode. Define all API response types in `src/lib/api.ts`. Do not use any new npm packages — use only the `fetch` API for HTTP and the existing dependencies in `package.json`.

What gets fixed, file by file

src/components/ui/Editor.tsx	Partially connected → Fully connected
• Fetches problem from GET /problem/{id} on mount	
src/components/ui/floating-dock.tsx	Partially connected → Fully connected
• Active route detection via usePathname()	
src/app/page.tsx	Not connected → Connected via layout
• Session start fires via layout.tsx on app load	
src/app/workspace/page.tsx	Shell → Full routing controller
• Manages selectedProblemId state	

New files to create

File	Responsibility
src/lib/api.ts	All API calls in one place. startSession, chat (streaming), getProblemList, getProblem, submitProblem, getWeeklyInsight.
src/components/ui/Chat.tsx	K■do chat panel. Session greeting on mount, streaming replies, action routing (open_problem), typing indicator.
src/components/ui/ProblemList.tsx	Fetches and renders all problems as cards. Difficulty badges. onSelect callback routes to Editor.
src/components/ui/WeeklyInsight.tsx	Fetches and displays weekly learning summary. Skeleton loader. Shown on workspace home screen.